

MICROCONTROLLERS

Labo 4

Naslag

Gegenereerd op 29/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

Bits en bytes

[Werken met bits](#)

I²C en de PCF8574

[Bibliotheken gebruiken](#)

[Werken met de PCF8574](#)

[Het I²C adres vinden](#)

Datasheets

PCF8574 los document

NASLAG

Werken met bits

De PCF8574 stuurt acht pinnen tegelijk aan met één byte. Je kan dus niet één pin apart aanspreken, want je leest en schrijft altijd alle acht samen. Om toch met één pin te kunnen werken heb je een paar operatoren nodig die je in de vorige labo's nog niet gebruikt hebt.

Een byte is acht bits

Een **byte** bestaat uit 8 bits, en elke bit is 0 of 1. Bij de PCF8574 hoort elke bit bij één pin. De **rechtse** bit is P0, de **linkse** is P7:

P7	P6	P5	P4	P3	P2	P1	P0
0	0	1	0	0	1	1	0

De byte B00100110 pin per pin

Die bit helemaal rechts noemen we **bit 0**, de volgende bit 1, en zo verder tot bit 7. De nummers van de bits komen dus overeen met de nummers van de pinnen.

Drie schrijfwijzen voor dezelfde waarde

Je mag een byte in je code op drie manieren schrijven. Ze betekenen exact hetzelfde, en de compiler maakt er geen onderscheid tussen:

```
1 byte a = B00100110; // binair: je ziet elke bit staan
2 byte b = 0x26;      // hexadecimaal
3 byte c = 38;        // decimaal
```

CPP

KIES DE SCHRIJFWIJZE DIE JE CODE LEESBAAR MAAKT

Gaat het over welke pin aan of uit staat, gebruik dan **binair**. In **B11111110** zie je meteen dat enkel P0 laag staat. In **0xFE** moet je dat eerst omrekenen. Gaat het over een gewoon getal, dan is decimaal duidelijker.

De operatoren

Deze vier operatoren werken **bit per bit**. Ze rekenen niet met de waarde van de byte als geheel, maar zetten bit 0 tegenover bit 0, bit 1 tegenover bit 1, enzovoort.

Operator	Naam	Wat ze doet
&	AND	Een bit in het resultaat is 1 als hij in <i>beide</i> operanden 1 is.
 	OR	Een bit in het resultaat is 1 als hij in <i>minstens één</i> van beide operanden 1 is.
~	NOT	Keert elke bit om: elke 0 wordt 1 en elke 1 wordt 0.
<<	schuiven naar links	Schuift alle bits een aantal plaatsen naar links. Rechts komen nullen bij.

De bitoperatoren die je in dit labo nodig hebt

⚠ VERWAR & NIET MET &&

`&&` is de logische EN die je in een `if` gebruikt: die kijkt naar twee voorwaarden als geheel en levert waar of niet waar op. `&` met één teken werkt bit per bit en levert een byte op. Datzelfde verschil bestaat tussen `|` en `||`. Eén teken te veel of te weinig compileert vaak gewoon, en de fout is dan moeilijk terug te vinden.

Eén bit uitlezen: maskeren

Je leest de hele byte in, maar je wil weten hoe het met één bepaalde pin gesteld is. Daarvoor gebruik je een **masker**: een byte waarin enkel de bit staat waarin je geïnteresseerd bent. Met `&` houd je van de ingelezen waarde alleen die ene bit over, want overal waar het masker 0 staat wordt het resultaat sowieso 0.

CPP

```
1 byte waarde = B01010101; // wat we van de chip lezen
2
3 bool bit0 = waarde & B00000001; // 1, want de rechtse bit van waarde is 1
4 bool bit1 = waarde & B00000010; // 0, want de tweede bit van waarde is 0
```

Uitgeschreven voor `bit1`:

waarde	0	1	0	1	0	1	0	1
masker	0	0	0	0	0	0	1	0
resultaat	0	0	0	0	0	0	0	0

B01010101 & B00000010

Het resultaat is 0, dus die bit stond laag. Merk op dat het resultaat van zo'n maskering niet 1 hoeft te zijn wanneer de bit hoog staat: `waarde & B00000010` levert bij een hoge bit de waarde 2 op, niet 1. Voor een `bool` of een `if` maakt dat niets uit, want alles wat niet 0 is telt als waar.

i BIJ EEN PULL-UPWEERSTAND IS INGEDRUKT EEN 0

Wil je expliciet weten of een knop ingedrukt is, schrijf dan de vergelijking er voluit bij. Dat leest een stuk duidelijker dan een omgekeerde `bool`:

```
bool knop1 = (waarde & B00000001) == 0;
```

CPP

Een masker berekenen met `<<`

Zolang je de maskers zelf uitschrijft is er niets aan de hand. Maar zodra je in een lus werkt, ken je het bitnummer pas terwijl je programma loopt. Dan bereken je het masker met `<<`: je vertrekt van een 1 helemaal rechts en schuift ze zoveel plaatsen naar links als je nodig hebt.

```
1 1 << 0 // B00000001, de bit van P0
2 1 << 1 // B00000010, de bit van P1
3 1 << 5 // B00100000, de bit van P5
```

CPP

`1 << n` is dus het masker van bit `n`, en daarmee kan je in een lus elke pin apart aanspreken.

Eén bit aanpassen zonder de rest te raken

Je schrijft altijd de hele byte naar de chip, dus wil je één bit veranderen en de zeven andere laten staan. Met OR zet je een bit op 1, met AND van het omgekeerde masker zet je hem op 0:

```
1 byte patroon = B11111111;
2
3 patroon = patroon | (1 << 2); // bit 2 op 1 zetten
4 patroon = patroon & ~(1 << 3); // bit 3 op 0 zetten
```

CPP

Die tweede regel lees je van binnen naar buiten. `1 << 3` is `B00001000`, het masker van bit 3. `~` keert dat om tot `B11110111`: overal een 1 behalve op de plaats van bit 3. Doe je daar een AND mee, dan blijft elke andere bit gewoon zichzelf, en wordt enkel bit 3 gegarandeerd 0.

Omdat je het resultaat meestal in dezelfde variabele terugstopt, mag je dat korter schrijven. `a &= b` betekent precies hetzelfde als `a = a & b`, net zoals `a += b` hetzelfde is als `a = a + b`:

```
1 patroon |= (1 << 2); // bit 2 op 1
2 patroon &= ~(1 << 3); // bit 3 op 0
```

Alle bits vergelijken in één keer

Een laatste patroon dat je in dit labo tegenkomt, is nagaan welke knoppen *net* ingedrukt zijn. Een knop die je ingedrukt houdt, blijft immers doorlopend een 0 geven, waardoor je programma tientallen keren per seconde een nieuwe druk registreert.

Bewaar daarvoor de vorige byte, en zoek de bits die van 1 naar 0 gegaan zijn:

```
byte netIngedrukt = ~knoppen & vorigeKnoppen;
```

~knoppen heeft een 1 staan op elke plaats waar een knop **nu** ingedrukt is. **vorigeKnoppen** heeft een 1 staan op elke plaats waar een knop **vorige keer** los was. Een AND van die twee houdt dus precies de knoppen over die deze doorloop van los naar ingedrukt gegaan zijn. Alle acht de knoppen worden daarbij in één bewerking behandeld.

TWIJFEL JE? SCHRIJF DE TABEL UIT

Bij twijfel helpt het altijd om de twee bytes onder elkaar te zetten, zoals in de tabellen hierboven, en de bewerking kolom per kolom te maken. Dat is trager dan het in je hoofd doen, maar je hebt het antwoord wel meteen juist.

Bibliotheken gebruiken

Eén van de sterke punten van de Arduino-omgeving is het grote aanbod aan softwarebibliotheken. Grote stukken ingewikkelde code zitten verpakt in een bibliotheek, zodat jij alleen nog de functies hoeft te kennen die ze aanbiedt.

We bekijken als voorbeeld de **Wire**-bibliotheek, waarmee je I²C-elektronica kan aansturen.

Een bibliotheek toevoegen

Om een bibliotheek te gebruiken moet ze eerst toegevoegd zijn aan de Arduino-omgeving. Wire is standaard al geïnstalleerd, dus die hoeft je niet apart te installeren. Andere bibliotheken voeg je het gemakkelijkst toe via de optie **Bibliotheken beheren**, te vinden via het menu **Schets > Bibliotheek gebruiken**. Via datzelfde menu kies je daarna de bibliotheek die je wil gebruiken.

Bovenaan je programma verschijnt dan een regel:

```
#include <naamVanDeBibliotheek.h>
```

CPP

In het geval van de Wire-bibliotheek wordt dat:

```
#include <Wire.h>
```

CPP

Uitzoeken welke functies er zijn

Welke functies een bibliotheek allemaal aanbiedt, vind je terug via de officiële Arduino-site. Via **Software > Reference** kom je op het overzicht van alle instructies. Onderaan die pagina staat een link naar de libraries-pagina, met een overzicht van de meeste bibliotheken. Kies je daar Wire, dan kom je op de specifieke pagina van die bibliotheek terecht en zie je rechts een lijst van de beschikbare functies.

Bibliotheekfuncties gebruiken

Door de `#include`-instructie zijn al die functies beschikbaar in je eigen programma. Voor het aansturen van een PCF8574 volstaat het bijvoorbeeld om in de setup een `Wire.begin()` op te nemen:

CPP

```
1 void setup()
2 {
3     Wire.begin();
4 }
```

En in de loop-functie de combinatie:

CPP

```
1 void loop()
2 {
3     Wire.beginTransaction(adres);
4     Wire.write(waarde);
5     Wire.endTransmission();
6 }
```

Elk van die functies moet de nodige parameters meekrijgen:

- `beginTransaction()` krijgt het **adres** mee van de chip waar je naartoe wil schrijven.
- `write()` krijgt de **waarde** mee die geschreven moet worden.
- `endTransmission()` zorgt ervoor dat de I²C-communicatie opgestart en uitgevoerd wordt.

LET OP DE HOOFDLETTERS

De functie heet `Wire.write()` met een kleine w. `Wire.Write()` bestaat niet en levert je een compileerfout op. Datzelfde geldt voor `Wire.read()` en `Wire.available()`.

Voorbeeldprogramma's

Elke goede bibliotheek (ja, er zijn er ook slechte) bevat een aantal voorbeeldprogramma's. Die kunnen vaak als leidraad dienen om te zien hoe je de functies het best implementeert. Je vindt ze terug via **Bestand > Voorbeelden > naam van de bibliotheek**.

Werken met de PCF8574

De PCF8574 is een I/O-expander: met twee pinnen van je Arduino krijg je er acht in- of uitgangen bij, en je kan er meerdere aan dezelfde bus hangen.

De I²C bus

De I²C bus bestaat uit twee lijnen: **SDA (Serial Data)** en **SCL (Serial Clock)**. Meestal worden ook **Vcc** en **GND** meegenomen, waardoor je in de praktijk een **4-lijns bus** krijgt.

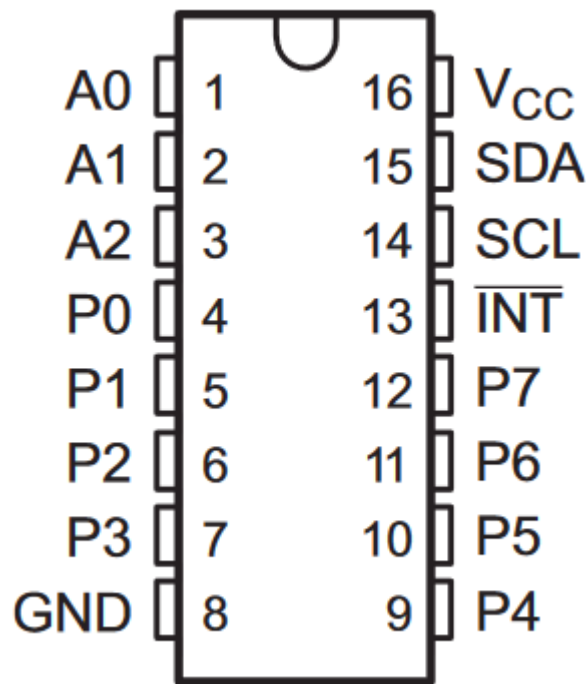
WAAR ZITTEN SDA EN SCL OP DE UNO?

Op de Arduino UNO liggen SDA en SCL vast: **SDA is A4** en **SCL is A5**. Op een UNO R3 zijn diezelfde twee lijnen ook nog eens apart uitgebracht naast de AREF-pin, dus je mag ze daar even goed aansluiten. Je kiest die pinnen niet zelf, want de I²C-hardware in de chip hangt eraan vast. Op een Leonardo, die je in sommige schema's hier ziet staan, zijn het pinnen 2 en 3.

Op deze bus kan je de PCF8574 aansluiten. **Per PCF8574 krijg je 8 extra I/O-lijnen**, en er kunnen tot 8 exemplaren van deze chip op dezelfde bus geplaatst worden. Met 2 pinnen van de Arduino beschik je dus over maximaal 64 extra in- en uitgangen.

Aansluitingen

We beginnen met 1 PCF8574, goed voor 8 extra ingangen en/of uitgangen. De aansluitingen zien er als volgt uit:



[Klik op de afbeelding om te vergroten](#)

De vier pinnen die op de bus aangesloten worden zijn Vcc en GND voor de voeding, en SDA en SCL voor de communicatie. De pinnen **P0 tot en met P7** zijn de 8 extra I/O-aansluitingen die we willen. Blijven nog over: A0 tot A2 en /INT.

Het adres van de chip

Elk I²C-component heeft een eigen ingebouwd adres dat het uniek maakt op de bus. Aan de hand van dat adres kan de master naar die component schrijven of ervan lezen.

Er zijn twee versies van de PCF8574. De ene heeft basisadres **0x20**, de andere (de A-versie, herkenbaar aan de **A** in de typenaam zoals PCF8574AN) heeft basisadres **0x38**. De lijnen **A2, A1 en A0** zijn verbonden met de **laagste 3 bits van dat adres**. Nemen we als voorbeeld de versie met basisadres 0x38, dan is het adres zo opgebouwd:

bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0	1	1	1	A2	A1	A0

Opbouw van het 7-bit adres van een PCF8574 met basisadres 0x38

Met die drie lijnen A2, A1 en A0 kan je 8 verschillende adressen instellen. Staan ze alle drie op 0, dan krijg je het basisadres 0x38. Zet je A0 op 1, dan heeft de chip adres 0x39. Door die drie lijnen elke mogelijke combinatie te geven kan je 8 PCF8574's op dezelfde bus gebruiken, goed voor 64 extra lijnen.

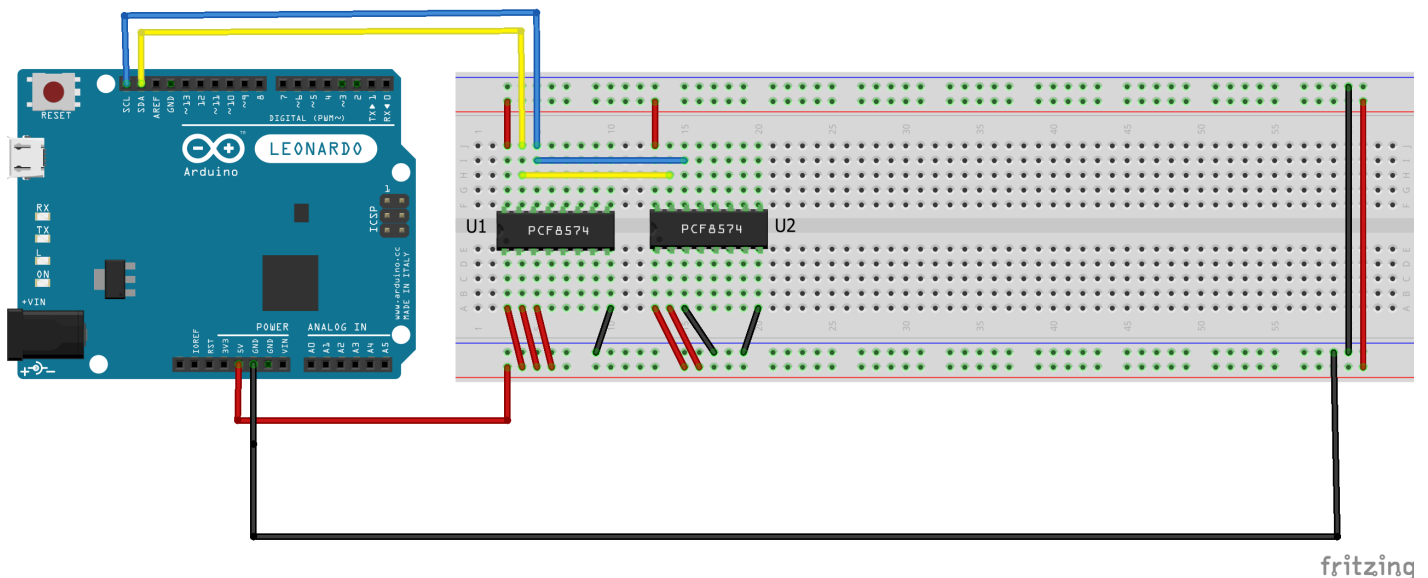
Gebruik je daarnaast ook nog de versie met basisadres 0x20, dan kan je er nog eens 8 chips bij plaatsen, wat het totaal op 128 extra I/O-lijnen brengt.

💡 LAAT DE ADRESLIJNEN NOOIT ZWEVEN

A0, A1 en A2 moeten altijd ergens aan vasthangen, aan GND of aan Vcc. Laat je ze los, dan is het adres van je chip niet voorspelbaar en vind je de chip soms wel en soms niet terug met de [I2C scanner](#).

Wie heb ik aan de lijn?

Op onderstaand schema staan twee PCF8574's op dezelfde bus. De I2C scanner vindt de adressen **0x3F** en **0x23**.



[Klik op de afbeelding om te vergroten](#)

De linkse chip is U1 en de rechtse U2. Welke van de twee heeft adres 0x23?

Aangezien er twee types PCF8574 bestaan, de ene met basisadres 0x38 en de andere met 0x20, is meteen duidelijk dat we hier één chip van elke soort hebben.

Bekijken we U1, dan zien we dat de drie adresbits (pinnen 1, 2 en 3) allemaal hoog staan. Bij het basisadres komt dus de waarde 7 bij. Die chip heeft dan als adres $0x38 + 0x07 (= 0x3F)$ of $0x20 + 0x07 (= 0x27)$. Daaruit besluiten we dat adres 0x3F bij U1 hoort, en dat het andere gevonden adres (0x23) dat van U2 moet zijn.

WAAROM STAAT ER EEN LEONARDO OP HET SCHEMA?

Dit schema komt uit het cursusmateriaal en is met een Leonardo getekend. Voor de PCF8574 maakt dat niets uit: het zijn dezelfde vier draden. Werk je met een UNO, sluit SDA dan aan op A4 en SCL op A5.

De /INT pin

De /INT-pin wordt **geactiveerd zodra er een wijziging optreedt op één of meer van de 8 I/O-lijnen**. Daardoor kan die pin dienen als **interruptsignaal** voor de master wanneer je de 8 lijnen als ingangen gebruikt. In plaats van constant via de bus te moeten checken of een pin van waarde veranderd is, krijg je via /INT een signaal.

In dit labo gebruiken we die functie nog niet. Ze komt terug in labo 7, waar je hem aan een interruptpin van je Arduino hangt en je expander je dus zelf komt melden dat er iets veranderd is. Zie [De PCF8574 laat van zich horen](#).

Aan de slag

We maken gebruik van de [Wire-bibliotheek](#). Bovenaan je programma staat dus de regel:

```
#include <Wire.h>
```

CPP

Vooraleer je de I²C kan gebruiken moet je het Wire-object initialiseren. Dat doe je in de `setup()`-functie:

```
Wire.begin();
```

CPP

Schrijven naar de bus

Het *schrijven* naar de I²C bus gebeurt in 3 stappen.

Eerst wordt de communicatie geïnitieerd:

```
Wire.beginTransmission(adres);
```

CPP

`adres` is het adres van de chip die je wil aanspreken. Wanneer A2, A1 en A0 op 0 staan gebruik je hier 0x38 (of 0x20, afhankelijk van je type).

Vervolgens geef je de waarde van de byte mee die op de uitgangen moet komen:

```
Wire.write(waarde);
```

CPP

waarde mag je in elk talstelsel schrijven. Wil je het bitpatroon 10101010 op de pinnen P7 tot P0 plaatsen, dan schrijf je 170 decimaal, 0xAA hexadecimaal of B10101010 binair.

De communicatie wordt **afgesloten** met:

```
Wire.endTransmission();
```

CPP

Lezen van de bus

Het *lezen* gebeurt op een net iets andere manier. Om een byte te lezen van de I/O-lijnen begin je met:

```
Wire.requestFrom(adres, aantal);
```

CPP

Het adres is dat van het IC. *aantal* geeft aan hoeveel bytes informatie je verwacht. Bij een PCF8574 is dat altijd 1, maar er bestaan ook I²C-sensoren die meerdere bytes na elkaar versturen.

Vervolgens kijk je of er een byte klaarstaat om van de PCF8574 naar de master te gaan:

```
1  if (Wire.available())
2  {
3      waarde = Wire.read();
4  }
```

CPP

⚠ EEN UITGANG DIE LAAG STAAT, LEES JE NIET

De pinnen van de PCF8574 zijn **quasi-bidirectioneel**: er is geen aparte instelling voor ingang of uitgang. Een pin die je wil *lezen*, moet in zijn uitgangslatch een 1 hebben staan. Bij het opstarten staan alle pinnen hoog, dus meestal merk je hier niets van. Maar heb je er eerder een 0 naar geschreven, dan lees je die 0 terug en lijkt je knop permanent ingedrukt. Schrijf in je *setup()* dus een 1 naar elke pin die je als ingang gebruikt.

Voor de volledige specificaties, zoals hoeveel stroom een uitgang kan leveren of opnemen, kan je terecht in de [datasheet](#).

Het I²C adres vinden

Onderstaand programma, de I2C scanner, loopt alle adressen van de bus af om te kijken welke chips er aanwezig zijn. Het adres van elke gevonden chip verschijnt in de seriële monitor.

⚠ ZET JE SERIËLE MONITOR EERST OP 115200 BAUD

Open eerst de seriële monitor en pas de snelheid aan zoals ze in de code staat. Doe je dat niet, dan krijg je alleen onleesbare tekens te zien en kan de Arduino IDE vastlopen.

De scanner

CPP

```
1  #include <Wire.h>
2
3  void setup()
4  {
5      Wire.begin();
6
7      Serial.begin(115200);
8      while (!Serial); // op een Leonardo: wacht tot de seriële monitor open is
9
10     Serial.println("\nI2C scanner");
11 }
12
13 void loop()
14 {
15     int aantalGevonden = 0;
16
17     Serial.println("Scannen...");
18
19     // Adres 0 en 127 zijn gereserveerd, die slaan we over
20     for (byte adres = 1; adres < 127; adres++)
21     {
22         // De scanner gebruikt de returnwaarde van endTransmission() om te zien
23         // of er op dit adres een chip antwoordt
24         Wire.beginTransmission(adres);
25         byte fout = Wire.endTransmission();
26
27         if (fout == 0)
28         {
29             Serial.print("I2C-toestel gevonden op adres 0x");
30             if (adres < 16)
31             {
32                 Serial.print("0");
33             }
34             Serial.println(adres, HEX);
35
36             aantalGevonden++;
37         }
38         else if (fout == 4)
39         {
40             Serial.print("Onbekende fout op adres 0x");
41             if (adres < 16)
42             {
43                 Serial.print("0");
44             }
45             Serial.println(adres, HEX);
46         }
47     }
48
49     if (aantalGevonden == 0)
```



```
50 {
51     Serial.println("Geen I2C-toestellen gevonden");
52 }
53 else
54 {
55     Serial.println("Klaar");
56 }
57
58 delay(5000); // wacht 5 seconden op de volgende scan
59 }
```

Hoe werkt dat?

De scanner doet voor elk adres alsof hij iets gaat versturen, maar stuurt niets mee.

`Wire.endTransmission()` geeft een getal terug dat vertelt hoe dat afgelopen is. Antwoordt er een chip op dat adres, dan is die returnwaarde 0. Antwoordt er niets, dan krijg je een andere waarde en weet je dat daar niets hangt.

Deze sketch test de standaard 7-bit adressen. Toestellen met een langer adres worden mogelijk niet correct herkend.



VIND JE NIETS?

Controleer dan eerst je bedrading. De meest voorkomende oorzaken zijn een vergeten GND-verbinding tussen Arduino en breadboard, SDA en SCL die omgewisseld staan, of adreslijnen A0 tot A2 die nergens aan vasthangen. Zie ook [Werken met de PCF8574](#).

Deze sketch komt uit het cursusmateriaal en circuleert in veel varianten, onder andere op het Arduino-forum. De oorspronkelijke auteur is niet gekend.